

PAI 1. IntegriDos - Verificadores de Integridad en la Transmisión para Entidad Financiera

1. Introducción y Contexto Empresarial

En este Proyecto de Aplicación Integrada (PAI) se utilizarán técnicas avanzadas para verificar la integridad, autenticidad y no repudio en el almacenamiento y la transmisión de datos a través de redes públicas no seguras como Internet. El objetivo central es blindar las comunicaciones frente a amenazas reales como ataques de *Man-in-the-Middle* (MitM), *Replay* (reintervención), debilidades en la derivación de claves y ataques de canal lateral de tiempo (*timing side-channel attacks*).

En esta edición, una entidad financiera *FinTech* ("SecBank") desea ofrecer servicios de transferencias bancarias entre clientes mediante una arquitectura distribuida cliente-servidor. Toda la interacción a nivel de aplicación debe garantizar la integridad sin confiar únicamente en la capa de transporte.

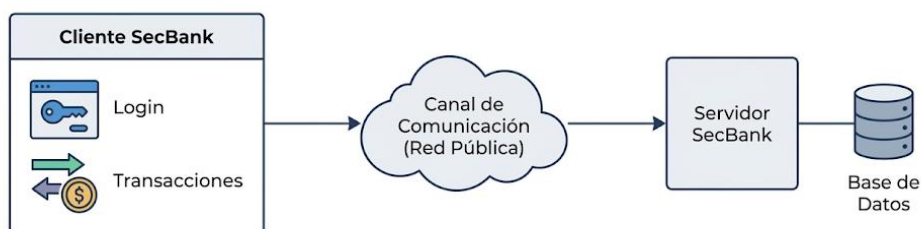


Figura 1: Arquitectura distribuida cliente-servidor

2. Política de Seguridad

La entidad financiera establece oficialmente la siguiente directiva dentro de su marco normativo de ciberseguridad:

"En todas las transacciones por medios electrónicos no seguros se debe conservar la integridad y autenticidad de las comunicaciones a nivel de aplicación. El servidor debe asegurar la integridad de la información de los usuarios registrados, de las sesiones activas y de cada una de las órdenes de transferencia procesadas."

3. Objetivos del proyecto

A continuación, se propone a los equipos de trabajo los siguientes objetivos:

1. Desarrollar un sistema cliente-servidor distribuido que permita el envío de credenciales y mensajes de transacciones bancarias en cumplimiento estricto con la Política de Seguridad.
2. Implementar en el servidor un mecanismo seguro de registro, almacenamiento y verificación de credenciales de usuario mediante funciones de derivación adaptativas y sal aleatoria.
3. Desarrollar un verificador de integridad para los mensajes de transferencia bancaria transmitidos por redes públicas, previniendo ataques MitM, Replay, Key Derivation y de Canal Lateral de Tiempo mediante el uso de firmas con HMAC, NONCEs y Timestamps, tamaños de clave adecuados y comparadores en tiempo constante.
4. Generar evidencias técnicas mediante capturas de tráfico de red (por ejemplo, archivos .pcap) y análisis de tiempo para auditar el comportamiento del sistema ante intentos de manipulación o reintervención.

4. Recomendaciones para el desarrollo

Arquitectura

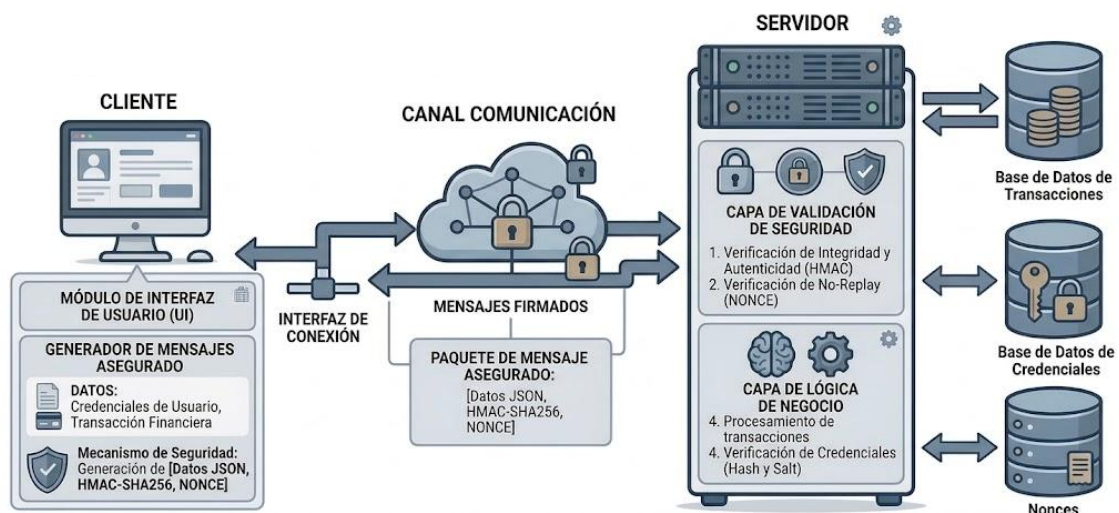


Figura 2: Arquitectura propuesta para la entidad financiera

Requisitos funcionales

1. **Gestión de Usuarios (Registro e Inicio de Sesión):**
 - a. Permitir el registro de nuevos usuarios con username y password.
 - b. Incluir una base de datos inicial con usuarios pre-registrados para pruebas.
 - c. Impedir el registro de usuarios duplicados.
 - d. Manejar sesiones activas e imponer un mecanismo de cierre de sesión (Logout).
2. **Transacciones:**
 - Una vez autenticado, el usuario puede realizar transacciones en formato **JSON** con la siguiente estructura:

```
{
  "tx_id": "UUIDv4",
  "origin_account": "ES1234567890123456789012",
  "destination_account": "ES9876543210987654321098",
  "amount": 1500.50,
  "currency": "EUR",
  "timestamp": 1741690000
}
```

- El servidor registrará la transacción únicamente tras comprobar de forma estricta su **integridad y autenticidad**.

Requisitos de seguridad

1. RS1. Almacenamiento y Verificación de Credenciales

- a. Derivación Robusta: Las contraseñas NO deben almacenarse en texto plano ni con funciones obsoletas como MD5/SHA1. Debe utilizarse un algoritmo de derivación de clave adaptativo (por ejemplo, Argon2id o PBKDF2-HMAC-SHA256) con un salt aleatorio único por usuario.
- b. Protección contra Fuerza Bruta: Implementar un mecanismo de *rate limiting* o bloqueo temporal de usuario tras N intentos fallidos consecutivos de login.

2. RS2. Garantía de Integridad y Autenticidad de Transacciones

- a. Firmas: Las transacciones transmitidas deben acompañarse de una firma MAC generada mediante, por ejemplo con HMAC-SHA256, utilizando una clave secreta derivada de la sesión o compartida.
- b. Longitud y Generación de Claves: Usar claves con un tamaño mínimo de 256 bits generadas mediante un PRNG criptográficamente seguro.

3. RS3. Protección contra Ataques de Replay (Reintervención)

- a. Nonces y Timestamps: Cada mensaje debe incluir un valor aleatorio único (NONCE) y/o un timestamp.
- b. Tabla de NONCES: El servidor debe mantener un registro de Nonces procesados dentro de una ventana temporal válida para rechazar cualquier mensaje duplicado.

4. RS4. Mitigación de Canales Laterales de Tiempo (Timing Attacks)

- a. Comparación en Tiempo Constante: La validación de códigos MAC en el servidor debe realizarse mediante funciones de comparación de tiempo constante (ej. `secrets.compare_digest()` en Python, `CRYPTO_memcmp()` en C/C++) para evitar la extracción de bytes válidos por diferencias de respuesta en microsegundos.

Verificación integridad y autenticación de mensajes

La recomendación para la implementación de la verificación de la integridad y la autenticación de transacciones en la transmisión de mensajes es usar el siguiente esquema que se muestra en la Figura 3.

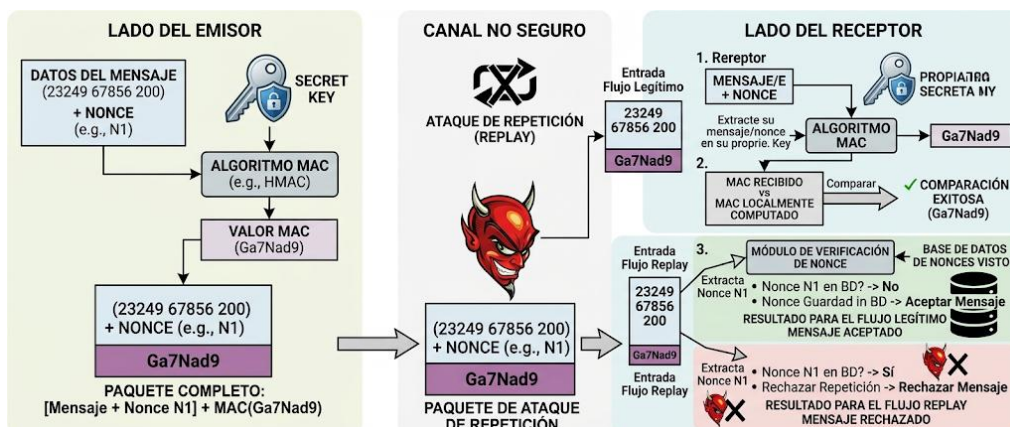


Figura 3: Esquema de ejemplo de verificación de integridad de mensajes.

Capa de transporte

Cada equipo de trabajo podrá seleccionar libremente la tecnología de comunicación cliente-servidor que mejor se adapte a su perfil de desarrollo (ver Figura 4) . Sin importar la opción elegida, **queda estrictamente prohibido el uso de cifrado o integridad en la capa de transporte (TLS, HTTPS o SSL) y será penalizado su uso**. Toda la lógica de protección frente a ataques de Man-in-the-Middle, Replay, derivación de claves y canales laterales debe implementarse manualmente en la capa de aplicación.

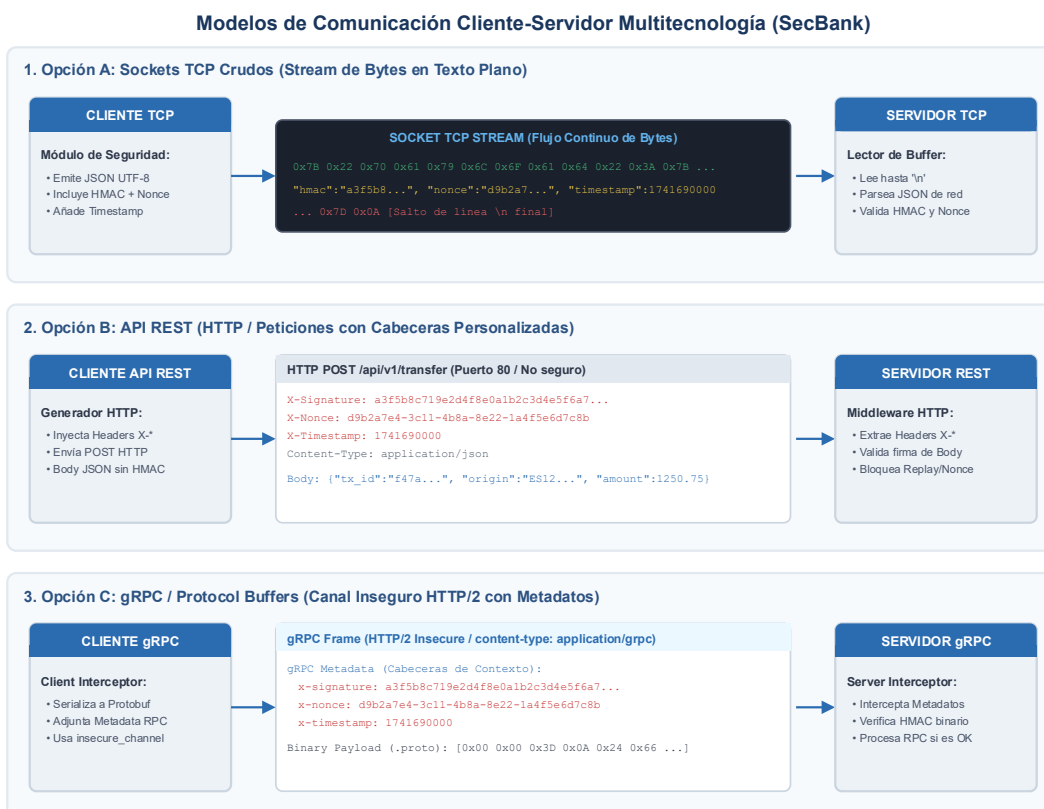


Figura 4: Opciones de implementación de capa de transporte

Se establecen tres modalidades tecnológicas equivalentes a efectos de evaluación:

- **Opción A: Sockets TCP Crudos (Enfoque Tradicional / Bajo Nivel)**
 - **Mecanismo:** Comunicación directa sobre sockets TCP utilizando las librerías nativas del lenguaje elegido (socket en Python, C, Java, Go, etc.).
 - **Formato de red:** Tramas JSON codificadas en UTF-8 delimitadas por un salto de línea (\n) o encabezadas por un entero de 4 bytes que indique la longitud de la carga útil (payload). Un ejemplo de trama JSON en este formato puede ser:

```
{
  "action": "TRANSFER",
  "payload": {
    "tx_id": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
    "origin_account": "ES1234567890123456789012",
    "destination_account": "ES9876543210987654321098",
    "amount": 1250.75,
    "currency": "EUR"
  },
  "timestamp": 1741690000,
  "nonce": "d9b2a7e4-3c11-4b8a-8e22-1a4f5e6d7c8b",
  "hmac":
"a3f5b8c719e2d4f8e0a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c
1d2e3"
}\n
```

- **Estructura de seguridad:** El objeto JSON transmitido incluye en su interior tanto los datos de la transacción como los campos de control (hmac, nonce, timestamp).
- **Opción B: API REST (Enfoque APIs Modernas)**
 - **Mecanismo:** Servidor HTTP no seguro (ej. FastAPI, Flask, Express, Spring Boot) procesando peticiones POST sobre HTTP en texto plano (puerto 80/8080).
 - **Formato de red:** El cuerpo de la petición (Body) incluye únicamente los datos de la transacción o credenciales en JSON. Un ejemplo de petición POST en este formato puede ser:

```
POST /api/v1/transfer HTTP/1.1
Host: api.secbank.com:8080
Content-Type: application/json
X-Signature:
a3f5b8c719e2d4f8e0a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3
X-Nonce: d9b2a7e4-3c11-4b8a-8e22-1a4f5e6d7c8b
X-Timestamp: 1741690000
{
  "tx_id": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "origin_account": "ES1234567890123456789012",
  "destination_account": "ES9876543210987654321098",
  "amount": 1250.75,
  "currency": "EUR"
}
```

- **Estructura de seguridad:** Los metadatos de protección se deben inyectar y validar mediante cabeceras HTTP personalizadas (Headers):

- X-Signature: Por ejemplo, firma HMAC-SHA256 del cuerpo de la petición.
- X-Nonce: Valor aleatorio único (UUIDv4) generado para la transacción.
- X-Timestamp: Marca de tiempo del momento de envío.
- **Opción C: gRPC / Protocol Buffers (Enfoque Cloud / Microservicios)**
 - Mecanismo: Comunicación gRPC utilizando esquemas *.proto* sobre canal HTTP/2 no cifrado (*insecure_channel*).
 - Formato de red: Serialización binaria mediante Protocol Buffers.

```
package secbank;
message TransferRequest {
    string tx_id = 1;
    string origin_account = ES1234567890123456789012;
    string destination_account = ES9876543210987654321098;
    double amount = 400;
    string currency = EUR;
}
```

- Estructura de seguridad: Los parámetros de verificación (hmac, nonce, timestamp) se adjuntan en los metadatos del canal gRPC (gRPC Metadata) e interceptados mediante Middleware/Interceptors tanto en el cliente como en el servidor.

La elección de la tecnología de transporte no influirá en la elección de los algoritmos de derivación de claves, la construcción del firmas, el registro de Nonces contra ataques de *Replay* y la comparación en tiempo constante serán evaluados con el mismo nivel de exigencia técnica en las tres modalidades.

Otras Consideraciones

- El proyecto permite elegir libremente entre tres tecnologías de comunicación: Sockets TCP Crudos, API REST (HTTP/1.1) o gRPC (Protocol Buffers sobre HTTP/2). Sin embargo, queda estrictamente prohibido el uso de cifrado en la capa de transporte (TLS/HTTPS); toda la seguridad debe implementarse de forma obligatoria en la capa de aplicación.
- Libertad de Lenguaje de Programación: Se puede utilizar cualquier lenguaje de programación (Python, Go, Java, C#, Rust, Node.js, etc.), siempre que cuente con soporte para el estándar criptográfico, funciones de derivación de claves seguras y el mecanismo de transporte seleccionado.
- El sistema de almacenamiento puede ser cualquier motor de base de datos o mecanismo de persistencia (SQLite, PostgreSQL, MySQL, base de datos clave-valor, etc.)
- La implementación debe permitir la captura de tráfico en local (localhost) o en red mediante Wireshark o tcpdump. Los archivos de traza (.pcap) generados son un entregable obligatorio para auditar el comportamiento de las tramas firmadas sobre el canal no seguro.
- **Uso Ético y Responsable de Inteligencia Artificial (IA):** Se permite el empleo de herramientas de IA generativa (como ChatGPT, Claude, GitHub Copilot, etc.) como asistente para la programación, depuración o estructuración del documento. No obstante, se aplican los siguientes criterios académicos:
 - El estudiante es el único responsable del contenido de la memoria y del código entregado. Debe comprender en profundidad la implementación y ser capaz de explicar o defender en detalle cualquier control criptográfico utilizado (comparación en tiempo constante, flujo de hashing, gestión de nonces).
 - **El uso de IA no justifica el incumplimiento de las restricciones. Debe revisarse cuidadosamente que el código generado o asistido por IA no introduzca por defecto librerías de capa de transporte segura (TLS), funciones hash obsoletas (MD5/SHA1) o comparadores de cadenas vulnerables a ataques de canal lateral (timing attacks).**
 - Se deberá incluir un breve apartado en la memoria justificando las herramientas de IA empleadas, indicando las tareas específicas para las que se utilizaron y cómo se validó la corrección de los resultados.

Consideraciones sobre la documentación

1. **Portada (título, grupo, integrantes, tabla de control).**
2. **Resumen Ejecutivo**
3. **Diseño del Protocolo:** Diagramas de secuencia detallando el flujo de paquetes entre cliente y servidor.
4. **Análisis de Seguridad y Evidencias:**
 - **Inyección de Ataque MitM:** Captura demostrativa donde un mensaje alterado en tránsito es rechazado por el servidor.
 - **Prueba de Ataque Replay:** Demostración de reenvío de un paquete capturado y su bloqueo por reutilización de Nonce/Timestamp.
 - **Verificación de Tiempo Constante:** Explicación técnica de cómo el código garantiza resistencia frente a ataques de canal lateral.
5. **Manual de Despliegue y Ejecución:** Instrucciones simples para arrancar el servidor y cliente de prueba.
6. **Matriz de Trazabilidad de Requisitos:** Tabla indicando qué línea/fichero implementa cada Requisito Funcional y de Seguridad.

Normas del entregable

- Cada Security Team debe entregar a través de la Plataforma de Enseñanza Virtual y en la **actividad** preparada para ello un archivo zip, nombrado **PAI1-STX.zip**, que deberá contener al menos los ficheros siguientes:
 - ✓ **Código fuente del proyecto que contenga la lógica, configuración y pruebas desarrolladas, así como los archivos de logs y evidencias oportunas para demostrar cumplimiento de las políticas indicadas.**
 - ✓ **Documentación (máximo 10 páginas).**
- El plazo de entrega de dicho proyecto finaliza el **día 5 de octubre a las 23:59 horas**.
- Los proyectos entregados fuera del plazo establecido serán considerados inadecuados por el cliente y por tanto entrarán en penalización por cada día de retraso de entrega de 10% del total, hasta agotarse los puntos.
- El cliente no aceptará envíos realizados por email, ni mensajes internos de la enseñanza virtual, ni correo interno de la enseñanza virtual. Toda entrega realizada por estos medios conllevará una penalización en la entrega del 10%.